

# Explainable Cervical Cell Classification: A Preprocessing-Enhanced Deep Learning Approach with Grad-CAM using SIPaKMeD - Progress Report #2

Ayşe Ece Akkurt  
*Electrical-Electronics Engineering*  
*Abdullah Gül University*  
Kayseri, Türkiye  
ayseece.akkurt@agu.edu.tr

Selin Nisa Akgöl  
*Electrical-Electronics Engineering*  
*Abdullah Gül University*  
Kayseri, Türkiye  
selinnisa.akgol@agu.edu.tr

Muhammet Talha Akşit  
*Electrical-Electronics Engineering*  
*Abdullah Gül University*  
Kayseri, Türkiye  
muhammettalha.aksit@agu.edu.tr

## I. PROGRESS 1 SUMMARY

In Progress 1, we focused on preparing the SIPaKMeD dataset and investigating possible preprocessing and segmentation approaches. The dataset was organized into separate training, validation, and test sets after resizing the images to a common resolution. Different preprocessing methods such as median filtering and CLAHE were applied and visually compared in order to determine whether image enhancement would be beneficial for later stages of the project. Based on the observations, CLAHE was found to improve the visibility of cellular structures more effectively, while median filtering had only a limited impact on image quality.

After preprocessing, several segmentation methods were investigated. Ground truth nucleus masks were generated using the coordinate annotations provided in the dataset, allowing segmentation results to be evaluated quantitatively. Otsu thresholding was initially tested but produced inconsistent results because of staining variations and background complexity. Therefore, alternative segmentation approaches based on GrabCut and K-Means clustering were explored. The performance of the generated nucleus masks was evaluated using the Dice similarity coefficient by comparing them with the ground truth masks. Our current direction of the project seems a little disconnected from our previous work, mainly because, as we experimented and explored new ways, we used the ones that our time permitted.

## II. MODEL TRAINING WITHOUT SEGMENTING NUCLEUS AND CYTOPLASM

### A. Feature Extraction & Selection

In the machine learning part of the project, the raw cell images were not given directly to the classifiers. Classical machine learning models such as SVM, Random Forest and KNN cannot understand image pixels in the same way as deep learning models. Therefore, before training the machine

learning models, we first converted each image into numerical features. This process is called feature extraction.

First, the dataset folder was selected from the prepared train, validation, and test folders. The code found the correct split folder and used the existing dataset split. The selected folder was `SIPaKMeD_split_224`, which included separate train, validation, and test folders. All images were resized to  $224 \times 224$  pixels before feature extraction so that every image had the same size. This was important because feature extraction should be performed under the same image size conditions for all samples.

For the training set, augmentation was also applied. In our setup, `AUG_PER_IMAGE = 1`, which means that for each original training image, one augmented version was created. These augmented images were not saved as new image files. They were only created temporarily in memory, and then their features were extracted. This was done to increase the amount of training data and help the machine learning models become more robust. The augmentation operations included small rotation, shifting, zooming, and horizontal flipping. Validation and test images were not augmented because they should represent clean unseen data.

As a result, the original training set had 2834 images. After adding one augmented version for each training image, the total number of training feature rows became 5668. The validation set had 607 rows and the test set had 608 rows. In total, the final feature table had 6883 rows. This means that each row represented one image or one augmented version of a training image.

After preprocessing and augmentation, 219 numerical features were extracted from each image. These features were grouped into different categories because different visual properties of the cell can be useful for classification.

The first group was color statistics. These features were extracted from RGB and HSV color spaces. For each channel, values such as mean, standard deviation, minimum, maximum,

TABLE I: Feature extraction dataset summary

| Dataset Part              | Number of Rows |
|---------------------------|----------------|
| Original training images  | 2834           |
| Augmented training images | 2834           |
| Total training rows       | 5668           |
| Validation rows           | 607            |
| Test rows                 | 608            |
| Total rows                | 6883           |

median, 25th percentile, and 75th percentile were calculated. These features describe the general color distribution of the cell image. For example, hue-related features such as `mean_H`, `median_H`, `q25_H`, and `q75_H` were useful because different cell classes may have different staining and color characteristics.

The second group was color histogram features. Instead of only using average color values, histograms describe how pixel values are distributed across different intensity ranges. RGB and HSV histograms were extracted with 8 bins per channel. These features are useful because two images may have similar average color values but different color distributions.

The third group was texture features. Texture is important in cervical cell classification because different cell types may have different internal patterns and surface appearances. For this reason, GLCM and LBP features were extracted. GLCM features measure texture properties such as contrast, dissimilarity, homogeneity, energy, correlation, ASM, and entropy at different directions. These directions were  $0^\circ$ ,  $45^\circ$ ,  $90^\circ$ , and  $135^\circ$ . LBP features were also extracted at multiple scales. LBP describes local texture patterns by comparing each pixel with its surrounding pixels.

The fourth group was shape and geometry features. These features were extracted after thresholding the grayscale image and finding the main contour. Area, perimeter, circularity, bounding box size, aspect ratio, extent, solidity, equivalent diameter, eccentricity, and Hu moments were calculated. These features describe the shape and structure of the detected cell region. Shape features are important because different cell classes may have different morphology.

The fifth group was grayscale intensity features. These features included grayscale mean, standard deviation, minimum, maximum, median, quartiles, and grayscale histogram bins. These values describe the brightness distribution of the image.

The last group was edge-based features. Canny edge count, edge density, Sobel mean, Sobel standard deviation, Sobel maximum, and Laplacian variance were extracted. These features describe how strong the edges and boundaries are in the image. This can be useful because cell boundaries, nucleus structures, and texture details may differ between classes.

After all these steps, the feature extraction process produced CSV files for the train, validation, test, and combined datasets. The train CSV contained both original and augmented training features, while the validation and test CSV files contained only original image features. This separation was important because augmentation should only affect the training process, not the evaluation process.

However, using all 219 features may not always be the best option. Some features may be very useful for separating the classes, while some may be weak, repetitive, or noisy. If too many unnecessary features are used, the model may become more complex and may overfit the training data. Therefore, feature selection was applied after feature extraction.

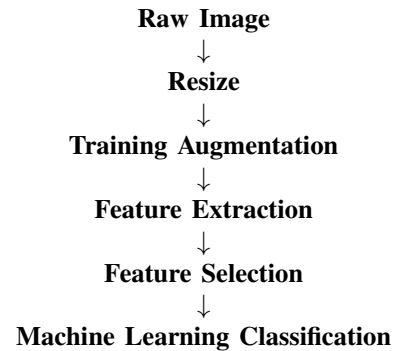
For feature selection, SelectKBest with Mutual Information was used. Mutual Information measures how strongly each feature is related to the class label. In simple terms, if a feature gives useful information about which class an image belongs to, it gets a higher score. If a feature does not help much in distinguishing the classes, it gets a lower score.

Using these Mutual Information scores, different feature subsets were created: Top 50, Top 100, Top 150, and All 219 features. The purpose of creating different feature sets was to compare whether the models performed better with fewer but more important features or with all extracted features.

The selected features were also grouped and explained in separate clean feature files. For example, in the Top 50 selected features, many of the highest-ranked features were color-related features such as `q25_H`, `q75_H`, `median_H`, `max_H`, `std_H`, and `min_H`. This shows that hue and color distribution were very important for separating the five cell classes. Edge features such as `sobel_std`, `sobel_max`, and `sobel_mean` were also highly ranked, which means that edge strength and boundary information were also useful. Texture features such as GLCM contrast, GLCM homogeneity, GLCM dissimilarity, and LBP histogram bins were also included among the selected features, showing that texture information contributed to classification.

In general, the feature selection results showed that color, texture, edge, intensity, and shape features all had different levels of importance. Color features were especially dominant in the top ranks, but texture and edge features also provided useful information. Therefore, feature selection helped us understand which types of image information were most useful for the classification task.

Overall, the machine learning pipeline for the non-segmented part followed this structure:



This approach made it possible to use classical machine learning models on image data. Instead of giving raw pixels directly to the models, each image was represented by meaningful numerical features. Then, the most informative features were selected and used for classification.

## B. Classical ML Models

In this section, classical machine learning models were trained by using the extracted numerical feature set. In this part, the models did not use raw cell images directly. Instead, they used numerical features extracted from the cell images. The main aim was to test whether classical machine learning models could classify the five cervical cell classes successfully.

First, all 219 extracted features were used for model training. Then, feature selection experiments were performed with the top 50, top 100, and top 150 features. This was done to understand whether a smaller number of selected features could give similar or better classification results. The same train, validation, and test split was used for all models. Therefore, the model comparison was fair.

Several classical machine learning models were tested: Support Vector Machine (SVM), Random Forest, K-Nearest Neighbors (KNN), Logistic Regression, Decision Tree, and Gaussian Naive Bayes. These models were selected because they use different classification methods. SVM tries to find a strong decision boundary between classes. Random Forest combines many decision trees to make a final decision. KNN classifies a sample by looking at the nearest samples. Logistic Regression was used as a simple linear baseline model. Decision Tree and Gaussian Naive Bayes were also tested to compare simpler classical methods.

The best classical machine learning result was obtained with the SVM model using all 219 features. This model achieved 94.90% test accuracy and a macro F1-score of 0.9492. This result shows that the full feature set contained useful information for cervical cell classification. Although the top 100 and top 150 feature sets also gave strong results, they did not improve the SVM performance. Therefore, using all 219 features gave the best result for the classical ML part.

Fig. 1 shows the confusion matrix of the SVM model using all 219 features. The model performed well for all five classes. The highest number of correct predictions was obtained for the Superficial-Intermediate class. Some errors were observed between Koilocytotic and Metaplastic samples. This means that these two classes may have similar feature patterns, so the model sometimes confused them.

Random Forest also achieved a strong result with all 219 features. It reached 93.91% test accuracy and a macro F1-score of 0.9393. This shows that tree-based ensemble models can also learn useful patterns from the extracted features. However, its performance was slightly lower than the SVM model. For this reason, SVM was selected as the best classical machine learning model in this experiment.

Table II shows the best result obtained from each classical machine learning model. The SVM model gave the highest accuracy and macro F1-score. Random Forest and KNN also gave strong results. Logistic Regression gave a moderate result. Decision Tree and Gaussian Naive Bayes had lower performance. This shows that SVM and Random Forest were more suitable for this feature set than the simpler models.

Overall, the classical machine learning results showed that the extracted numerical features were useful for cervical cell

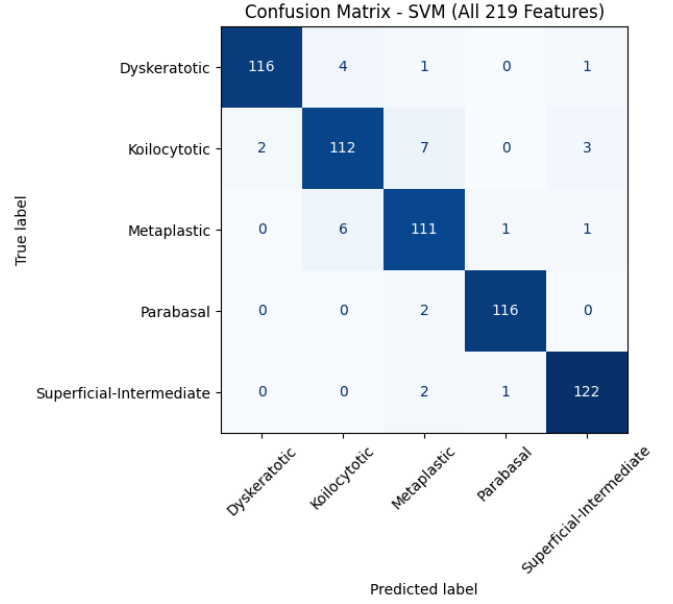


Fig. 1: Confusion matrix of the SVM model using all 219 extracted features.

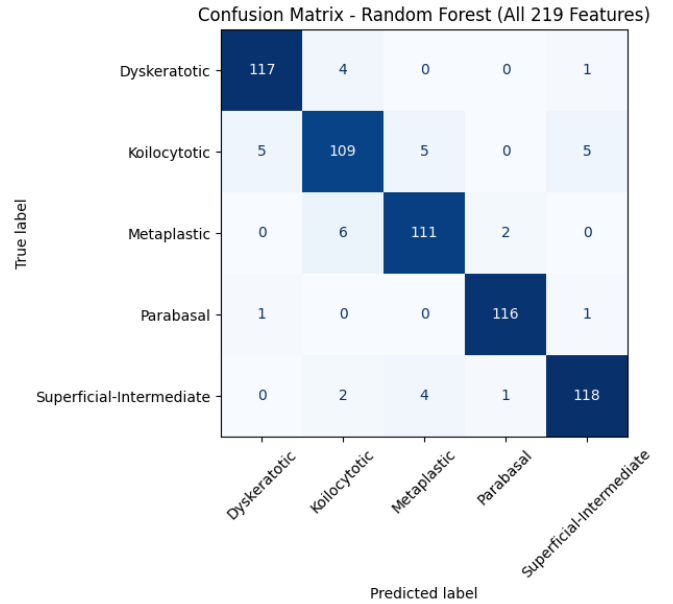


Fig. 2: Confusion matrix of the Random Forest model using all 219 extracted features.

TABLE II: Best classical machine learning results for each model.

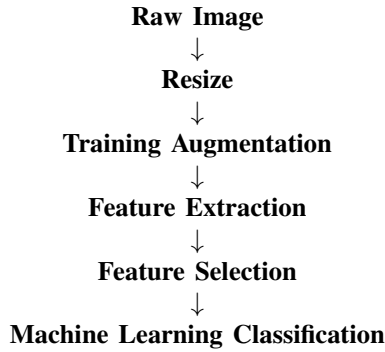
| Model                | Best Feature Set | Accuracy | Macro F1 | Weighted F1 | Features |
|----------------------|------------------|----------|----------|-------------|----------|
| SVM                  | All Features     | 0.9490   | 0.9492   | 0.9491      | 219      |
| Random Forest        | All Features     | 0.9391   | 0.9393   | 0.9390      | 219      |
| KNN                  | Top 150          | 0.9359   | 0.9359   | 0.9356      | 150      |
| Logistic Regression  | Top 150          | 0.9260   | 0.9263   | 0.9260      | 150      |
| Decision Tree        | All Features     | 0.8783   | 0.8792   | 0.8787      | 219      |
| Gaussian Naive Bayes | All Features     | 0.7632   | 0.7654   | 0.7641      | 219      |

classification. Among all tested classical models, SVM with all 219 features gave the best result. This means that the full feature set was more effective than the selected smaller feature sets for the SVM model. However, when this result was compared with the segmented-feature deep learning models, the classical ML accuracy was lower. Therefore, classical machine learning models provided a strong baseline, but deep learning models using segmented nucleus-cytoplasm features achieved better performance.

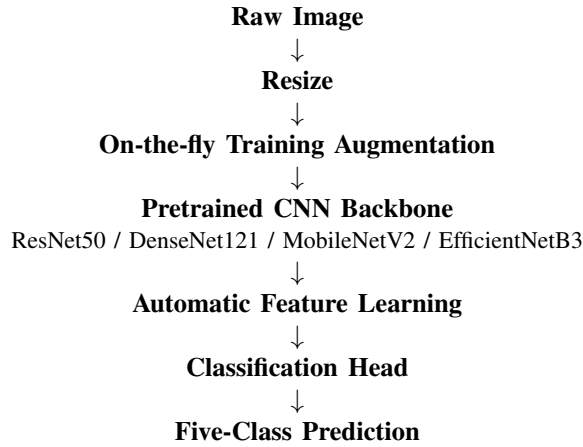
### C. Deep Learning Models

In the non-segmented deep learning part, the original cervical cell images were used directly as input to pretrained CNN models. Unlike the classical machine learning pipeline, manual feature extraction and feature selection were not applied. This is because convolutional neural networks learn visual features automatically through convolutional layers.

In the classical machine learning part, the workflow was:



However, in the deep learning part, the CNN backbone itself performed feature extraction. Therefore, the workflow became:



Here, the CNN backbone works like an automatic feature extractor. Early layers usually learn simple visual patterns such as edges, colors, corners, and basic textures. Middle layers learn more complex visual patterns, and deeper layers learn more class-specific features. Therefore, feature extraction was done automatically by the CNN models. There was also no separate manual feature selection step like SelectKBest. Instead, the network learned useful visual patterns by updating its weights during training.

1) *On-the-Fly Data Augmentation*: Data augmentation was applied only to the training set. In our project, this was done on-the-fly, meaning that augmented images were generated dynamically during training and were not permanently saved as new image files.

The augmentation operations included small rotations, width and height shifts, zooming, and horizontal flipping. These operations were used because cervical cell images may naturally appear in slightly different positions, angles, and scales. By using augmentation, the model became less dependent on the exact appearance of the training images and learned more general patterns.

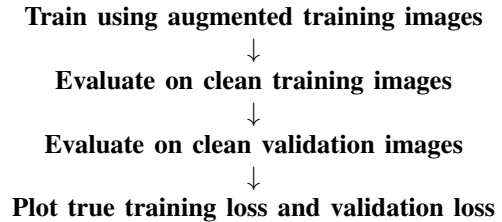
Validation and test images were not augmented. This was important because validation and test sets should represent clean unseen data. If augmentation were also applied to validation or test images, the evaluation results would be harder to interpret.

2) *Correction of Training and Validation Loss Curves*: During the first experiments, validation loss sometimes appeared lower than training loss. At first, this looked wrong. However, this situation can happen in deep learning when training-time conditions are harder than validation-time conditions. Rosebrock explains that validation loss may be lower than training loss because of data augmentation, dropout, regularization, and the difference between training and inference behavior [1].

In our case, the original training loss was calculated on augmented training batches. These training images were harder because they were rotated, shifted, zoomed, or flipped. However, validation loss was calculated on clean validation images. Therefore, the two loss values were not measured under exactly the same conditions.

To make the loss curves more meaningful, augmentation was still kept during training, but the plotted training loss was recalculated at the end of each epoch using the clean non-augmented training images. In this way, both training loss and validation loss were measured on clean images. This did not remove augmentation from training. It only corrected how the loss curves were reported.

The corrected loss calculation can be summarized as:



Because of this correction, the final loss curves became easier to understand. Instead of comparing augmented training loss with clean validation loss, we compared clean training loss with clean validation loss.

3) *General Transfer Learning Strategy*: For all CNN models, transfer learning was used. Instead of training the models from scratch, pretrained ImageNet weights were loaded. This was useful because the SIPaKMeD dataset is much smaller

than ImageNet. Pretrained models already learned general visual patterns such as edges, colors, textures, and shapes.

The training strategy had two main stages:

- 1) **Feature extraction stage:** The pretrained CNN backbone was frozen, and only the newly added classification head was trained.
- 2) **Fine-tuning stage:** The final high-level block of the CNN backbone was unfrozen and trained with a smaller learning rate.

The reason for freezing the backbone at the beginning was to protect the pretrained weights. If all layers were trained from the start, the pretrained visual information could be damaged. After the classification head learned the new five-class task, the last block was opened for fine-tuning.

Only the last blocks were opened because early CNN layers learn general features, while deeper layers learn more task-specific features. Therefore, fine-tuning only the upper layers allowed the model to adapt to cervical cell images without destroying the general pretrained knowledge.

Batch Normalization layers were kept frozen during fine-tuning. These layers contain running statistics, and updating them on a relatively small medical dataset may make training unstable. Therefore, freezing Batch Normalization helped keep the pretrained behavior stable.

4) *ResNet50*: ResNet50 was one of the main pretrained CNN models used in this project. It uses residual connections, which help train deeper networks more effectively.

For ResNet50, the first stage was feature extraction. In this stage, the ResNet50 backbone was frozen and only the new classification head was trained. The classification head was responsible for converting the CNN features into five cervical cell classes.

After this stage, fine-tuning was applied by opening the final conv5 block. Earlier blocks were kept frozen because they contain general features such as edges and textures. The conv5 block was opened because it contains higher-level features that are more related to the target classification task.

TABLE III: ResNet50 training strategy

| Stage              | Trainable Layers                  | Learning Rate      |
|--------------------|-----------------------------------|--------------------|
| Feature Extraction | Classification head only          | $1 \times 10^{-4}$ |
| Fine-Tuning        | conv5 block + classification head | $1 \times 10^{-5}$ |

The corrected loss curves showed stable learning. During feature extraction, both training and validation losses decreased. During fine-tuning, the model improved further without severe overfitting.

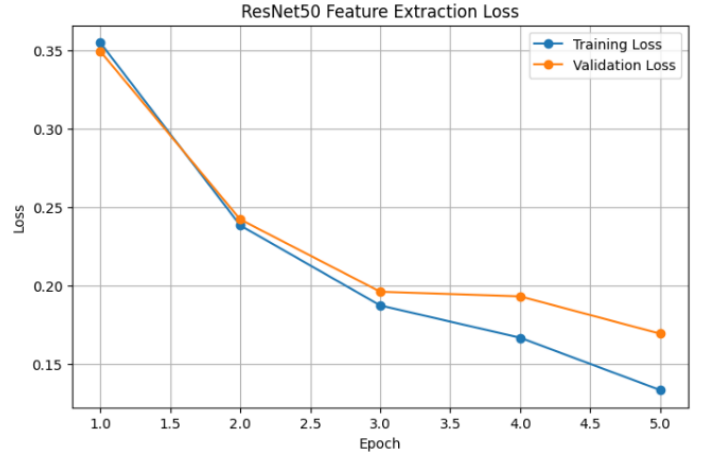


Fig. 3: ResNet50 Feature Extraction Loss Graph

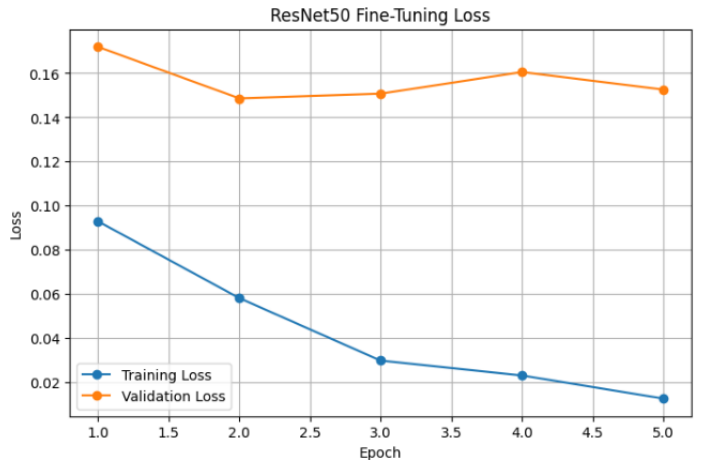


Fig. 4: ResNet50 Fine Tuning Loss Graph

TABLE IV: Fine-Tuned ResNet50 Classification Report

| Class                    | Precision | Recall | F1-score | Support |
|--------------------------|-----------|--------|----------|---------|
| Dyskeratotic             | 0.94      | 0.97   | 0.95     | 122     |
| Koilocytotic             | 0.91      | 0.84   | 0.87     | 124     |
| Metaplastic              | 0.88      | 0.95   | 0.91     | 119     |
| Parabasal                | 0.97      | 0.99   | 0.98     | 118     |
| Superficial-Intermediate | 0.99      | 0.94   | 0.96     | 125     |
| Accuracy                 |           |        | 0.94     | 608     |
| Macro avg                | 0.94      | 0.94   | 0.94     | 608     |
| Weighted avg             | 0.94      | 0.94   | 0.94     | 608     |

Table IV presents the classification performance of the fine-tuned ResNet50 model on the test dataset. The model achieved an overall accuracy of 94%, indicating that most cervical cell images were classified correctly. The highest performance was obtained for the Parabasal class with an F1-score of 0.98, showing that this cell type was identified very successfully. Similarly, the Dyskeratotic and Superficial-Intermediate classes achieved high F1-scores of 0.95 and 0.96, respectively. The Koilocytotic class had the lowest F1-score (0.87), mainly due to its lower recall value, which means that some Koilocytotic cells were confused with other classes. However, the overall Macro F1-score and Weighted

F1-score were both 0.94, demonstrating balanced classification performance across all five classes. These results show that fine-tuning the last ResNet50 block helped the model learn discriminative features specific to cervical cell images and improved its classification capability.

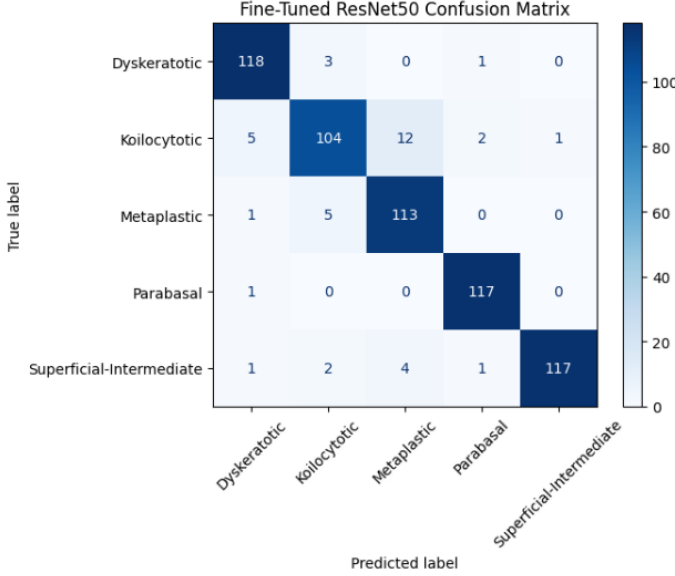


Fig. 5: ResNet50 Confusion Matrix

TABLE V: ResNet50 baseline and fine-tuned performance comparison

| Stage            | Train  | Val    | Test   | Acc.   | Macro F1 | Ep. |
|------------------|--------|--------|--------|--------|----------|-----|
| Baseline Frozen  | 0.1336 | 0.1695 | 0.2007 | 0.9293 | 0.9288   | 5   |
| Fine-Tuned conv5 | 0.0127 | 0.1526 | 0.1808 | 0.9359 | 0.9358   | 10  |

The fine-tuned ResNet50 model achieved a test accuracy of 93.59% and a Macro F1-score of 0.9358. This result was better than the frozen baseline ResNet50 model. This shows that opening the final conv5 block helped the model adapt better to cervical cell images.

5) *DenseNet121*: DenseNet121 was also trained using transfer learning. DenseNet uses dense connections, where each layer receives information from previous layers. This helps feature reuse and supports stable training.

The same two-stage strategy was applied. First, the DenseNet121 backbone was frozen and only the classification head was trained. Then, during fine-tuning, the final dense block was opened. In our implementation, this corresponded to the conv5 related layers.

The reason was the same as in ResNet50. Lower layers contain general visual features, while the last block contains more dataset-specific features. By opening only the final block, the model could adapt to cervical cell morphology and texture without changing the whole network.

TABLE VI: DenseNet121 training strategy

| Stage              | Trainable Layers                         | Learning Rate      |
|--------------------|------------------------------------------|--------------------|
| Feature Extraction | Classification head only                 | $1 \times 10^{-4}$ |
| Fine-Tuning        | Final dense block / conv5 related layers | $1 \times 10^{-5}$ |

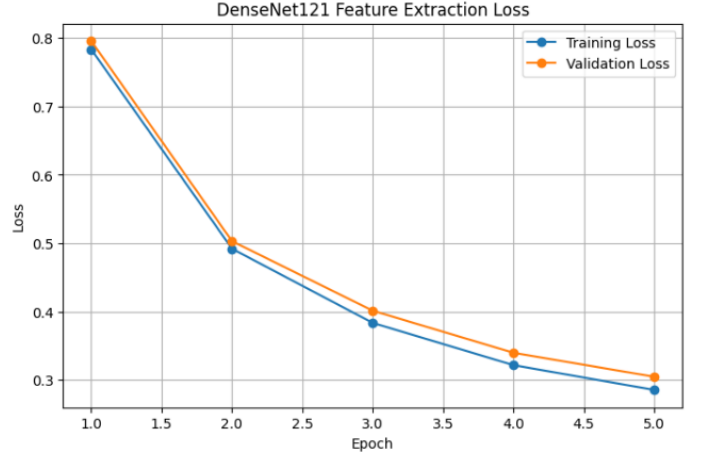


Fig. 6: DenseNet121 Feature Extraction Loss Curve

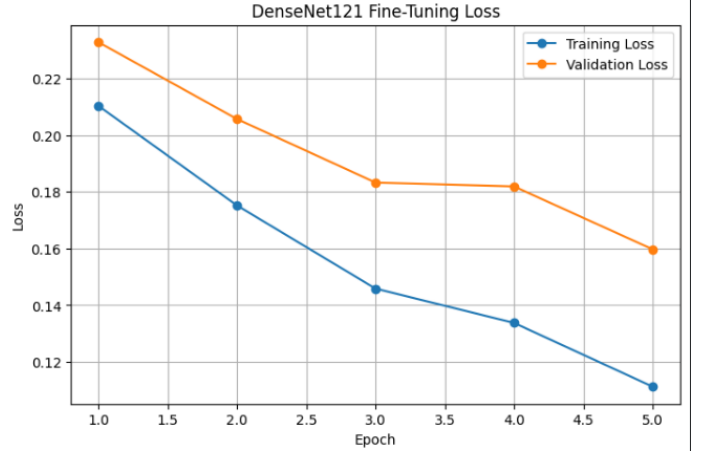


Fig. 7: DenseNet121 Fine-Tuning Loss Curve

TABLE VII: DenseNet121 baseline and fine-tuned performance comparison

| Stage            | Train  | Val    | Test   | Acc.   | Macro F1 | Epoch |
|------------------|--------|--------|--------|--------|----------|-------|
| Baseline Frozen  | 0.2854 | 0.3047 | 0.3093 | 0.9112 | 0.9100   | 5     |
| Fine-Tuned conv5 | 0.1112 | 0.1597 | 0.1682 | 0.9342 | 0.9339   | 10    |

DenseNet121 also showed stable learning behavior. The final DenseNet121 model achieved a test accuracy of 93.42% and a Macro F1-score of 0.9339. Its result was very close to ResNet50.

TABLE VIII: Fine-Tuned DenseNet121 Classification Report

| Class                    | Prec. | Recall | F1   | Support |
|--------------------------|-------|--------|------|---------|
| Dyskeratotic             | 0.94  | 0.95   | 0.94 | 122     |
| Koilocytotic             | 0.90  | 0.84   | 0.87 | 124     |
| Metaplastic              | 0.89  | 0.92   | 0.90 | 119     |
| Parabasal                | 0.98  | 0.97   | 0.98 | 118     |
| Superficial-Intermediate | 0.96  | 0.99   | 0.98 | 125     |
| Accuracy                 | —     | —      | 0.93 | 608     |
| Macro avg                | 0.93  | 0.93   | 0.93 | 608     |
| Weighted avg             | 0.93  | 0.93   | 0.93 | 608     |

Table VIII presents the classification performance of the fine-tuned DenseNet121 model on the test dataset. The model achieved an overall accuracy of 93%, indicating that most cervical cell images were classified correctly. The highest F1-scores were obtained for the Parabasal and Superficial-Intermediate classes (0.98), showing that these cell types were identified very successfully. The Dyskeratotic class also achieved strong performance with an F1-score of 0.94. The lowest F1-score was observed for the Koilocytotic class (0.87), mainly because its recall value was lower than the other classes, suggesting that some Koilocytotic cells were misclassified as different cell types. However, the Macro F1-score and Weighted F1-score were both 0.93, demonstrating balanced performance across all five classes. Overall, the results indicate that DenseNet121 was able to learn meaningful visual features from cervical cell images and achieved reliable classification performance after fine-tuning.

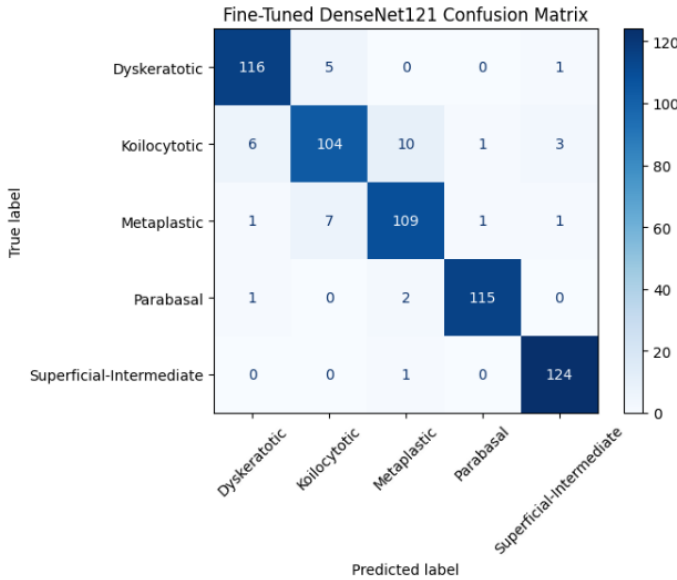


Fig. 8: DenseNet121 Confusion Matrix

The confusion matrix further confirms the effectiveness of the fine-tuned DenseNet121 model. Most samples are concentrated along the main diagonal, indicating correct classifications. The majority of classification errors occurred between Koilocytotic and Metaplastic cells, which is reasonable because these classes may share similar visual characteristics. Despite these misclassifications, the model maintained strong

performance across all classes and demonstrated good generalization ability on unseen test images.

6) *MobileNetV2*: MobileNetV2 was used as a lightweight CNN model. It has fewer parameters than ResNet50 and DenseNet121, so it is more efficient and faster. This makes it useful when computational resources are limited.

For MobileNetV2, the same transfer learning strategy was followed. First, the backbone was frozen and only the classification head was trained. Then, in the fine-tuning stage, the final high-level blocks were opened. In our implementation, `block_15` and `block_16` were made trainable.

These blocks were selected because they are near the end of the MobileNetV2 architecture. Earlier layers were kept frozen because they contain general visual information. Opening only `block_15` and `block_16` allowed the model to adapt to the cervical cell dataset while keeping the model stable.

TABLE IX: MobileNetV2 training strategy

| Stage              | Trainable Layers                                                          | Learning Rate      |
|--------------------|---------------------------------------------------------------------------|--------------------|
| Feature Extraction | Classification head only                                                  | $1 \times 10^{-4}$ |
| Fine-Tuning        | <code>block_15</code> ,<br><code>block_16</code> +<br>classification head | $1 \times 10^{-5}$ |

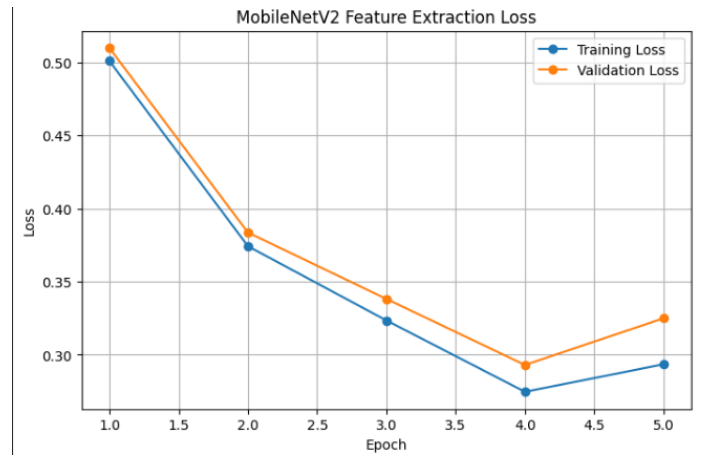


Fig. 9: MobileNetV2 Feature Extraction Loss Curve



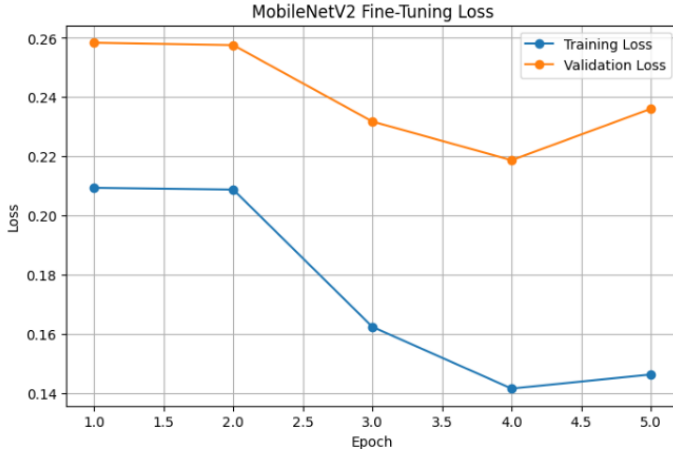


Fig. 10: MobileNetV2 Fine-Tuning Loss Curve

TABLE X: MobileNetV2 baseline and fine-tuned performance comparison

| Stage                           | Train  | Val    | Test   | Acc.   | Macro F1 | Ep. |
|---------------------------------|--------|--------|--------|--------|----------|-----|
| Baseline Frozen                 | 0.2934 | 0.3249 | 0.3038 | 0.8964 | 0.8956   | 5   |
| Fine-Tuned<br>block_15/block_16 | 0.1463 | 0.2360 | 0.2319 | 0.9161 | 0.9156   | 10  |

MobileNetV2 achieved a test accuracy of 91.61% and a Macro F1-score of 0.9156. This was lower than the other models. The reason is most likely that MobileNetV2 is designed to be lightweight, so it has less feature extraction capacity compared to larger models. However, it still produced a reasonable result.

TABLE XI: Fine-Tuned MobileNetV2 Classification Report

| Class                    | Prec. | Recall | F1   | Support |
|--------------------------|-------|--------|------|---------|
| Dyskeratotic             | 0.90  | 0.96   | 0.93 | 122     |
| Koilocytotic             | 0.86  | 0.88   | 0.87 | 124     |
| Metaplastic              | 0.92  | 0.86   | 0.89 | 119     |
| Parabasal                | 0.96  | 0.89   | 0.93 | 118     |
| Superficial-Intermediate | 0.95  | 0.99   | 0.97 | 125     |
| Accuracy                 | —     | —      | 0.92 | 608     |
| Macro avg                | 0.92  | 0.92   | 0.92 | 608     |
| Weighted avg             | 0.92  | 0.92   | 0.92 | 608     |

Table XI presents the classification report of the fine-tuned MobileNetV2 model. The model achieved an overall accuracy of 92% and a Macro F1-score of 0.92, showing that it performed well on the test dataset. The highest F1-score was obtained for the Superficial-Intermediate class with 0.97. Dyskeratotic and Parabasal classes also achieved good results with F1-scores of 0.93. The Koilocytotic class had the lowest F1-score of 0.87, which means this class was more difficult for the model to classify correctly.

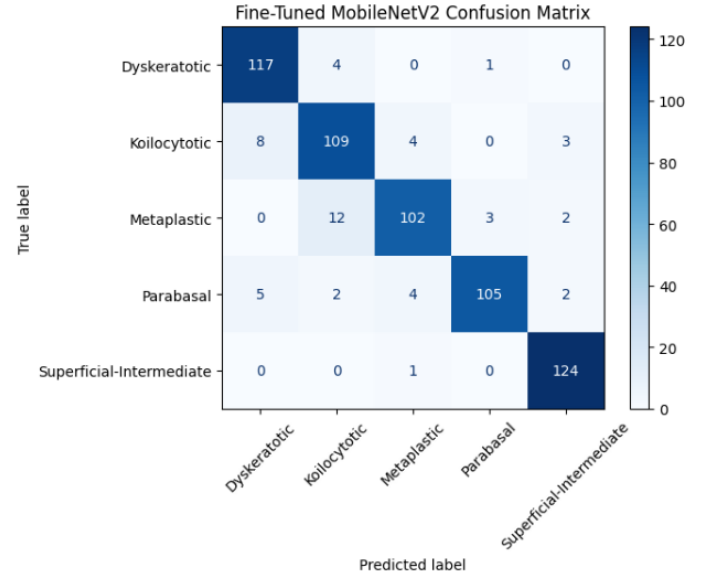


Fig. 11: MobileNetV2 Confusion Matrix

Figure 11 shows the confusion matrix of the fine-tuned MobileNetV2 model. Most predictions are located on the main diagonal, which means that the majority of test images were classified correctly. The main confusion occurred between Koilocytotic and Metaplastic cells, where some samples were predicted as the other class. This suggests that these two classes may have visually similar characteristics.

7) *EfficientNetB3*: EfficientNetB3 was used because EfficientNet models balance model depth, width, and input resolution. Compared with MobileNetV2, EfficientNetB3 has stronger feature learning capacity.

For EfficientNetB3, the image size was  $300 \times 300$  instead of  $224 \times 224$ . This was done because EfficientNetB3 is normally designed for a larger input resolution. A larger input image can help the model capture more detailed visual information from the cervical cell images.

The normal EfficientNetB3 training also had two stages. First, the backbone was frozen and only the classification head was trained. Then, block6, block7, and top\_conv were opened during fine-tuning.

TABLE XII: Normal EfficientNetB3 training strategy

| Stage              | Trainable Layers         | Learning Rate      |
|--------------------|--------------------------|--------------------|
| Feature Extraction | Classification head only | $1 \times 10^{-4}$ |
| Fine-Tuning        | block6, block7, top_conv | $1 \times 10^{-5}$ |



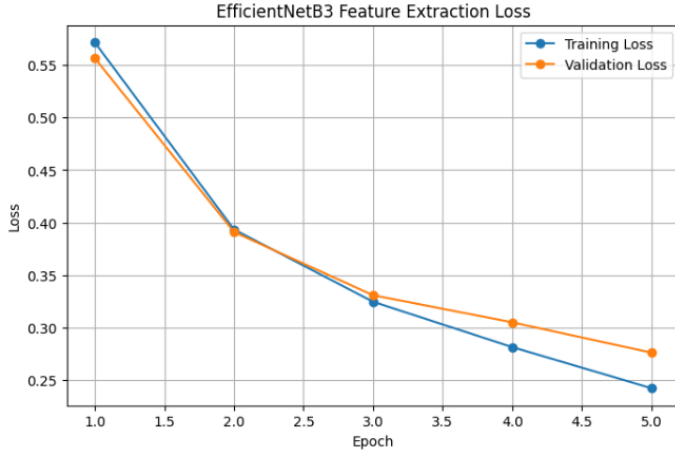


Fig. 12: EfficientNetB3 Feature Extraction Loss Curve

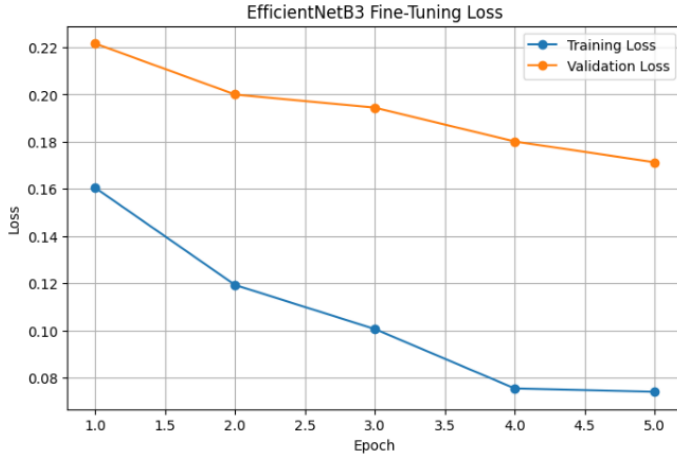


Fig. 13: EfficientNetB3 Fine-Tuning Loss Curve

TABLE XIII: EfficientNetB3 performance comparison

| Stage    | Train  | Val    | Test   | Acc.   | Macro F1 | Ep. |
|----------|--------|--------|--------|--------|----------|-----|
| Baseline | 0.2422 | 0.2761 | 0.3183 | 0.8914 | 0.8904   | 5   |
| FT-1     | 0.0741 | 0.1713 | 0.1982 | 0.9326 | 0.9321   | 10  |
| FT-2     | 0.0272 | 0.1538 | 0.1716 | 0.9457 | 0.9454   | 15  |

FT-1: block6/block7 fine-tuning. FT-2: extra fine-tuning with block5/block6/block7.

The normal fine-tuned EfficientNetB3 model achieved a test accuracy of 93.26% and a Macro F1-score of 0.9321. This was close to ResNet50 and DenseNet121, but it was not the best result.

Because EfficientNetB3 still had potential for improvement, an extra fine-tuning experiment was performed. In this extra experiment, block5 was also opened together with block6, block7, and top\_conv. This gave the model more flexibility to adapt to the cervical cell dataset.

When more layers are opened, the risk of damaging pre-trained weights increases. Therefore, the learning rate was reduced from  $1 \times 10^{-5}$  to  $5 \times 10^{-6}$ . This allowed the model to update more slowly and safely.

TABLE XIV: Extra EfficientNetB3 fine-tuning strategy

| Stage              | Trainable Layers                          | Learning Rate      |
|--------------------|-------------------------------------------|--------------------|
| Normal Fine-Tuning | block6,<br>block7,<br>top_conv            | $1 \times 10^{-5}$ |
| Extra Fine-Tuning  | block5,<br>block6,<br>block7,<br>top_conv | $5 \times 10^{-6}$ |

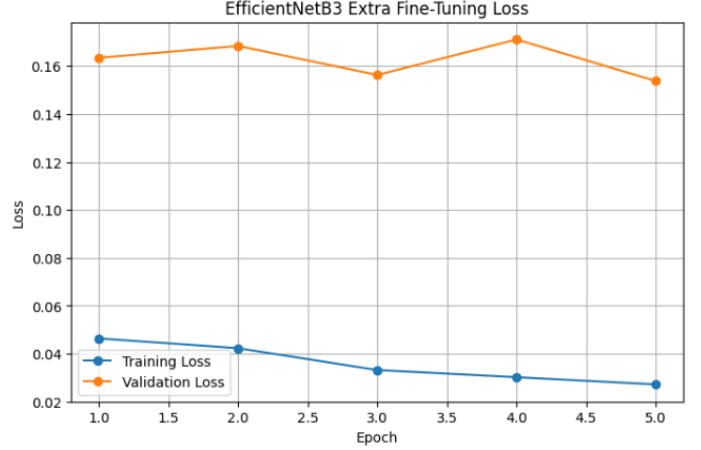


Fig. 14: EfficientNetB3 Extra Fine-Tuning Loss Curve

The extra fine-tuning improved the test accuracy from 93.26% to 94.57%. This showed that opening block5 helped the model learn more dataset-specific high-level features. Because the learning rate was reduced, the extra adaptation did not damage the pretrained weights.

TABLE XV: Extra Fine-Tuned EfficientNetB3 Classification Report

| Class                    | Prec. | Recall | F1   | Support |
|--------------------------|-------|--------|------|---------|
| Dyskeratotic             | 0.96  | 0.89   | 0.92 | 122     |
| Koilocytotic             | 0.91  | 0.93   | 0.92 | 124     |
| Metaplastic              | 0.94  | 0.93   | 0.94 | 119     |
| Parabasal                | 0.94  | 0.99   | 0.97 | 118     |
| Superficial-Intermediate | 0.97  | 0.99   | 0.98 | 125     |
| Accuracy                 | —     | —      | 0.95 | 608     |
| Macro avg                | 0.95  | 0.95   | 0.95 | 608     |
| Weighted avg             | 0.95  | 0.95   | 0.95 | 608     |

Table XV presents the classification performance of the extra fine-tuned EfficientNetB3 model. The model achieved an overall accuracy of 95% and a Macro F1-score of 0.95, which were the highest results obtained among all tested deep learning models. The best class-level performance was observed for the Superficial-Intermediate class with an F1-score of 0.98, followed by the Parabasal class with an F1-score of 0.97. The lowest F1-score was obtained for the Dyskeratotic and Koilocytotic classes (0.92), but the performance remained

strong across all classes. The high and balanced precision, recall, and F1-score values indicate that the model learned representative features for all five cervical cell categories.

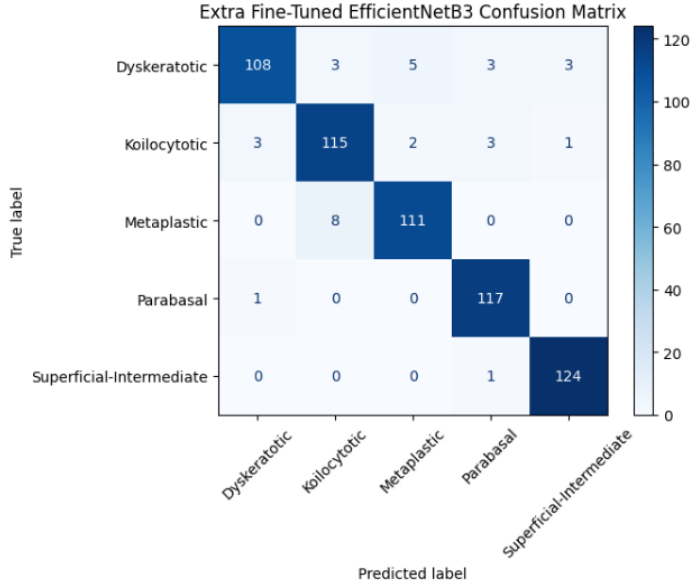


Fig. 15: EfficientNetB3 Confusion Matrix

Figure 15 shows the confusion matrix of the extra fine-tuned EfficientNetB3 model. Most samples are concentrated along the main diagonal, indicating a high number of correct classifications. Only a small number of misclassifications can be observed between Dyskeratotic, Koilocytotic, and Metaplastic cells. The Parabasal and Superficial-Intermediate classes were classified almost perfectly, with very few prediction errors.

8) *Final Deep Learning Model Comparison:* The final results are shown in the comparison table below. Accuracy shows the general correct classification rate, while Macro F1-score is important because it gives equal importance to each class.

TABLE XVI: Final comparison of deep learning models on non-segmented images

| Model       | Stage    | Train  | Val    | Test   | Acc.   | F1     | Ep. |
|-------------|----------|--------|--------|--------|--------|--------|-----|
| EffNetB3    | Extra FT | 0.0272 | 0.1538 | 0.1716 | 0.9457 | 0.9454 | 15  |
| ResNet50    | FT       | 0.0127 | 0.1526 | 0.1808 | 0.9359 | 0.9358 | 10  |
| DenseNet121 | FT       | 0.1112 | 0.1597 | 0.1682 | 0.9342 | 0.9339 | 10  |
| EffNetB3    | FT       | 0.0741 | 0.1713 | 0.1982 | 0.9326 | 0.9321 | 10  |
| MobileNetV2 | FT       | 0.1463 | 0.2360 | 0.2319 | 0.9161 | 0.9156 | 10  |

According to the final results, the best model was EfficientNetB3 Extra Fine-Tuned, with a test accuracy of 94.57% and a Macro F1-score of 0.9454. ResNet50 and DenseNet121 also performed very well and achieved close results. MobileNetV2 had the lowest score among the tested models, but it was still useful as a lightweight model.

### III. MODEL TRAINING WITH SEGMENTING NUCLEUS AND CYTOPLASM

#### A. Segmentation Method

Segmenting nucleus and cytoplasm regions is important for various reasons. When pathologists observe PAP smears, they

look at area, borders, nucleus to cytoplasm ratio, density of the nucleus etc. to identify the cell type. For this reason, we believe that segmenting them will increase the accuracy of our classifier model. The segmentation pipeline is as follows:

- 1) Read images & apply median filter (no CLAHE because it reduced the success of this segmentation method)
- 2) Transfer from RGB to LAB and only use L channel. Due to chromatic differences, only pixel intensity will be used
- 3) Normalize L channel to bring it to the 0-255 range. Although the L channel is already represented in the range of 0–255 in OpenCV, min-max normalization was applied to stretch the intensity distribution of each image to the full dynamic range. This reduces illumination variations between samples and improves the separability of intensity clusters during K-means segmentation.
- 4) Apply 20 layer k-means clustering to increase the success of segmenting the nucleus and pick a certain number of darkest layers (nucleus is usually the darkest part of the image) that increase the dice score of the nucleus mask and the ground truth masks. The number of layers are chosen after a trial of ascending combinations from the darkest 5 layers.
- 5) Keep the closest piece of mask to the center (nucleus is usually close to the center) and clean up the rest by opening and closing operations.
- 6) Evaluate the k-means mask with Dice and IoU.
- 7) Construct background mask from the lightest layer.
- 8) Construct the alternative ellipse mask in case the k-means fails because it usually fails for cytoplasm. Finds the biggest contour and fits an ellipse with `cv2.fitEllipse()`. Calculate Dice.
- 9) Compare the dice score for both the k-means and the ellipse mask. Whichever scores higher Dice will be chosen as the mask.
- 10) Remove nucleus and background masks. The cluster layers left will test for combinations with highest Dice score.
- 11) Fit ellipse mask to the failed k-means mask. Evaluate Dice score.
- 12) Compare the dice score for both the k-means and the ellipse mask. Whichever scores higher Dice will be chosen as the mask.

The masks are finalized after these steps and saved to a folder in .dat format for mask coordinates like the ground truth and .png for segmented images ready to be used in deep learning models. An example of segmentation is displayed in the Figure 16.

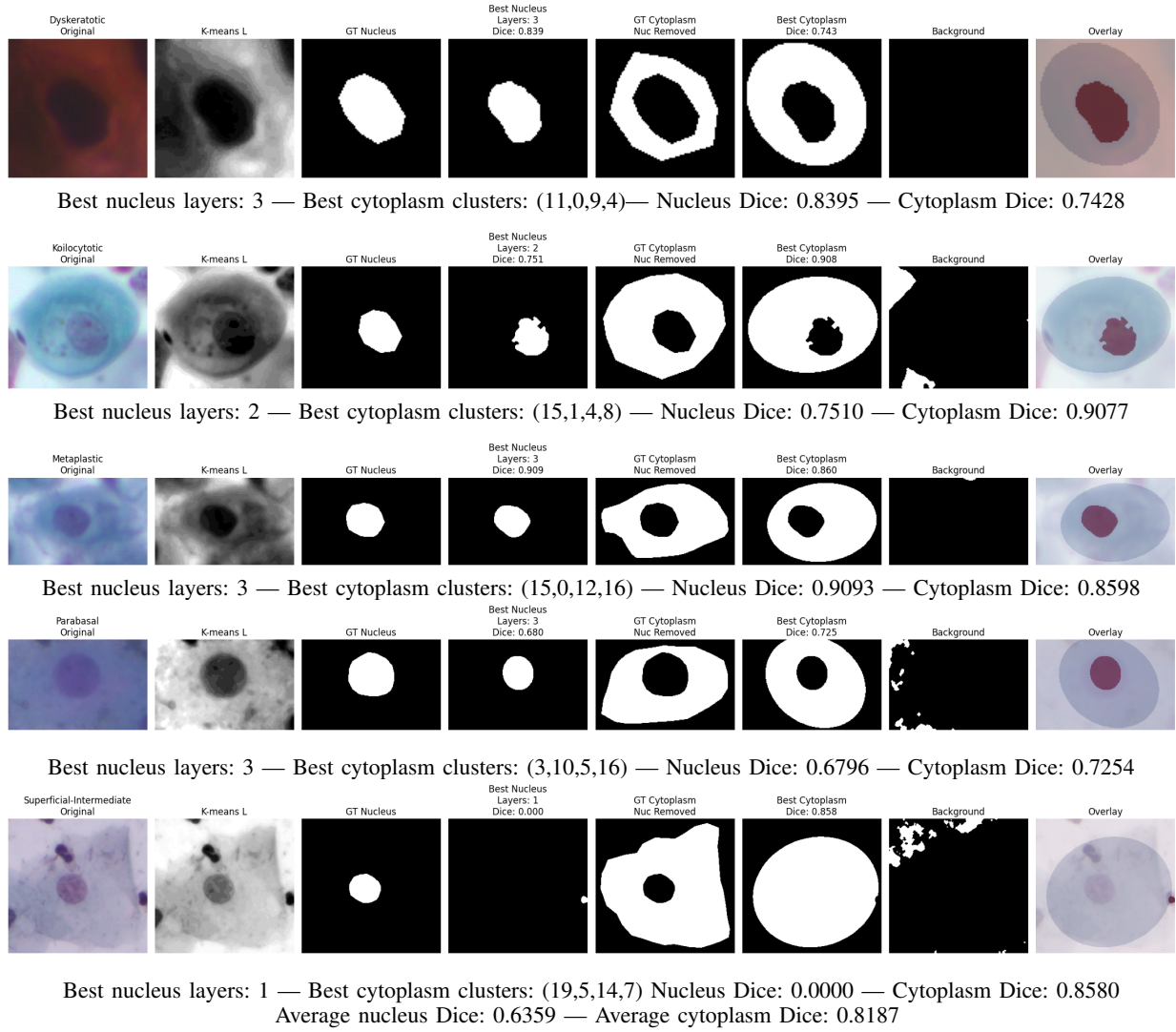


Fig. 16: Segmentation method applied on random images from all classes

After applying this method on the entire dataset:

- Average nucleus Dice: 0.8056447400701903
- Average cytoplasm Dice: 0.8456395563589089

### B. Feature Extraction & Selection

After the segmentation masks are finalized, the features are extracted. The morphological features, color features from color spaces (RGB, HSV, LAB), texture features with GLCM and LBP methods, edge features from the grayscale, are calculated and extracted for both masks. In addition, nucleus to cytoplasm ratio features are extracted. A total of 716 features are collected and ready to be used to train classical ML models. But before that the features are listed by descending mutual information score.

Mutual information score carries the information of how much the said feature helps to classify the cell correctly. 0 means no relation and 1 means absolute relation.

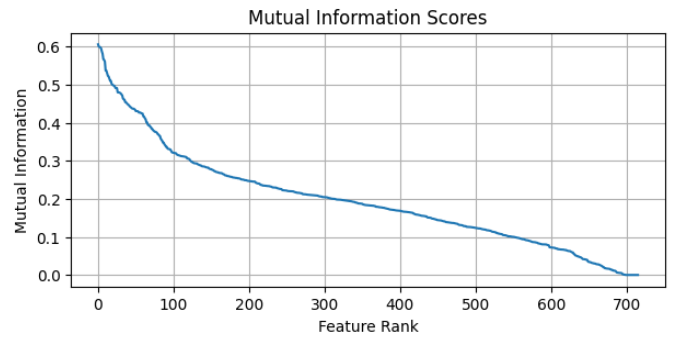


Fig. 17: Mutual score vs. number of features

### C. Classical ML Models

To keep the results consistent, the same ML models are used in here which are KNN, logistic regression (lr), gaussian Naive Bayes (gNB), SVM, Random Forest (RF) and Decision Tree (DT) with 70-15-15 train-val-test split. Every model is

tested at top 50, 100, 150 and all (716) features. At the end, a comparison table is built in descending order of accuracy. The results for highest performing models are presented in this subsection.

TABLE XVII: Performance comparison of the top 5 classical machine learning models.

| Model                               | Accuracy | Macro F1 | Macro Precision | Macro Recall |
|-------------------------------------|----------|----------|-----------------|--------------|
| Random Forest<br>All Features       | 0.9737   | 0.9736   | 0.9736          | 0.9737       |
| Random Forest<br>Top 150 Features   | 0.9688   | 0.9687   | 0.9690          | 0.9687       |
| SVM<br>All Features                 | 0.9655   | 0.9656   | 0.9659          | 0.9656       |
| KNN<br>All Features                 | 0.9589   | 0.9588   | 0.9588          | 0.9590       |
| Logistic Regression<br>All Features | 0.9572   | 0.9573   | 0.9575          | 0.9574       |
| SVM<br>Top 150 Features             | 0.9572   | 0.9572   | 0.9579          | 0.9573       |

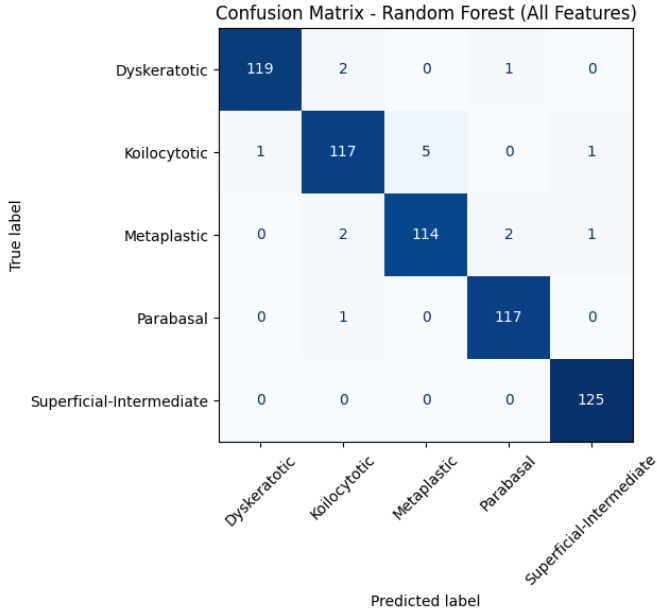


Fig. 18: Random Forest (All 716 Features) Confusion Matrix

There are a clear improvements to the same models with segmentation. Compared to all features (219) random forest without segmentation (0.9391 accuracy), the 150 features segmented version (0.9688 accuracy) performs better even while the feature number of the segmented model is higher than the non-segmented RF. These results perform this well even though the segmentation is not that successful with its Dice scores around 0.80, indicating that the improvement of segmentation can increase the accuracy of the models furthermore.

#### D. Deep Learning Models

In this section, deep learning models were trained by using the segmented nucleus-cytoplasm feature table. In the previous image-based model, the raw cell images were used directly. However, in this part, the model did not use the original images as input. Instead, each cell image was represented with 716 numerical features. These features were extracted from the segmented nucleus and cytoplasm regions.

The main reason for using segmented features was to give the models more meaningful cell information. In cervical cell classification, the nucleus and cytoplasm are important parts of the cell. Their size, shape, texture, and intensity values can help separate different cell classes. Therefore, this part aimed to test whether deep learning models could classify the cells better when they used these segmented numerical features.

Before training, all numerical features were standardized. This step was necessary because the feature values had different ranges. If the features are not scaled, some large-valued features may affect the model more than others. The complete dataset included 4049 cell images. After preprocessing, each image was represented by 716 numerical features. The dataset was then divided into training, validation, and test sets. The training set included 2834 samples, the validation set included 607 samples, and the test set included 608 samples. In total, these three sets correspond to all 4049 images. The same split was used for all models. This made the model comparison fair.

Four deep learning models were tested in this part: Basic MLP, Deep MLP, 1D CNN, and Autoencoder + MLP. The Basic MLP and Deep MLP models used fully connected layers. These models tried to learn the relation between the 716 segmented features and the five cell classes. The 1D CNN model used the feature vector as a one-dimensional input. This model was tested to see whether convolution layers could learn useful local patterns between the features. In the Autoencoder + MLP model, the autoencoder first compressed the 716 features into a smaller feature representation. Then, this compressed representation was used by an MLP classifier.

All models were trained with categorical cross-entropy loss and the Adam optimizer. Early stopping was used during training. This was done to stop the training when the validation loss did not improve. Therefore, it helped reduce the risk of overfitting. Model checkpointing was also used to save the best model during training.

At first, the validation loss was sometimes lower than the training loss in the segmented deep learning experiments. This looked confusing, but it was not caused by image augmentation because the segmented part used extracted feature CSV files, not raw image augmentation. The main reason was that dropout and training-time behavior were active while the model was learning, but they were inactive during validation. According to Rosebrock [1], validation loss can appear lower than training loss when the model behaves differently during training and validation. To make the loss curves easier to interpret, we calculated the training loss again at the end of each epoch using `model.evaluate()` on the training

feature set. This means the training loss and validation loss were both measured in evaluation mode. After this change, the loss curves became more reliable and easier to compare.

The Basic MLP model gave the best result among all tested models. Fig. 19 shows the true training and validation loss curves of this model. The training loss decreased quickly and became very low. The validation loss also decreased at the beginning, but after some epochs it stayed higher than the training loss. This means that the model learned the training data very well. However, the gap between training loss and validation loss shows a small generalization gap. In other words, the model fitted the training data more strongly than the validation data.

This behavior may indicate a slight overfitting tendency. For this reason, early stopping and model checkpointing were used during training. Early stopping helped stop the training when the validation loss did not improve. Model checkpointing saved the best model according to validation performance. Although the loss curve showed a small gap between training and validation, the Basic MLP still achieved high test accuracy and macro F1-score. Therefore, the model was considered successful on unseen test data.

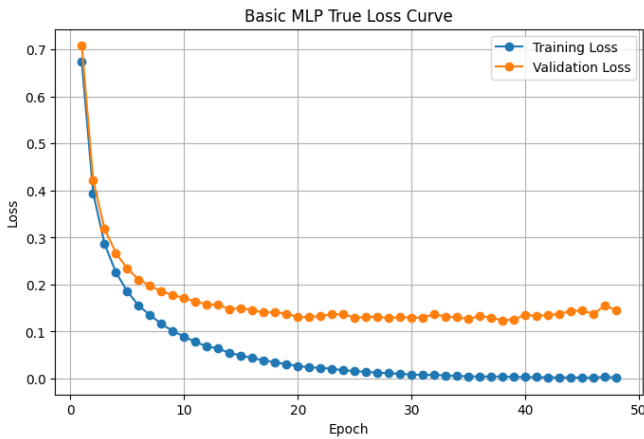


Fig. 19: True training and validation loss curves of the Basic MLP model using segmented nucleus-cytoplasm features.

Fig. 20 shows the confusion matrix of the Basic MLP model. The model classified all Parabasal samples correctly. It also classified almost all Superficial-Intermediate samples correctly. Most errors were seen between Dyskeratotic and Koilocytotic samples, and between Koilocytotic and Metaplastic samples. This means that these classes have similar feature patterns. For this reason, the model sometimes confused them.

The Deep MLP model was also tested to observe whether a deeper fully connected architecture could improve the classification performance. This model achieved 96.22% test accuracy, which was close to the Basic MLP and 1D CNN results. However, it did not outperform the Basic MLP model. This result shows that increasing the number of dense layers did not provide a clear improvement for this feature set. Therefore, the simpler Basic MLP model was preferred as the best segmented-feature deep learning model.

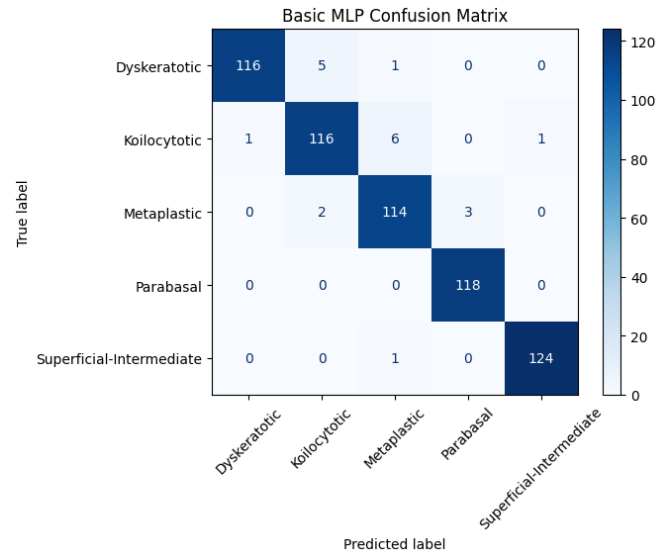


Fig. 20: Confusion matrix of the Basic MLP model for five-class cervical cell classification using segmented nucleus-cytoplasm features.

The 1D CNN model also gave a strong result. This model was tested because CNN layers can learn local relations between neighboring feature values. Fig. 21 shows the loss curve of the 1D CNN model. The training and validation losses decreased strongly in the first epochs. After this point, the training loss continued to decrease and became very low, while the validation loss stayed higher and became more stable. This result shows that the model learned the training data very strongly. However, the higher validation loss shows that the model did not fit the validation data as strongly as the training data. This can be interpreted as a slight overfitting tendency. Even with this gap, the 1D CNN model achieved high test accuracy, so it still generalized well on unseen test data.

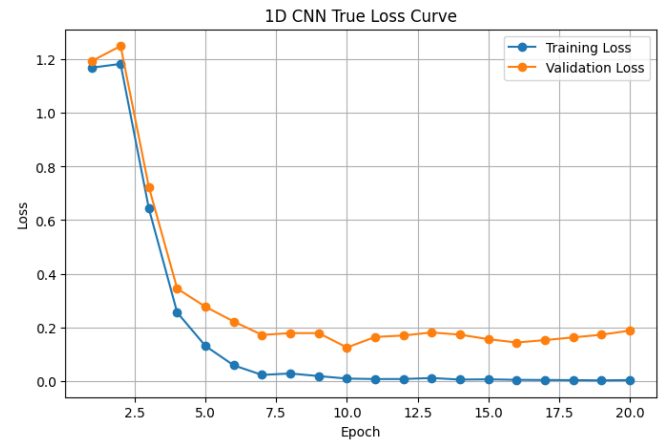


Fig. 21: True training and validation loss curves of the 1D CNN model trained on segmented tabular features.

The confusion matrix of the 1D CNN model is given in



Fig. 22. The model performed very well for Parabasal and Superficial-Intermediate classes. Similar to the Basic MLP model, some errors were observed between Koilocytotic and Metaplastic samples. This result shows that the segmented features are useful, but some classes are still close to each other in feature space.

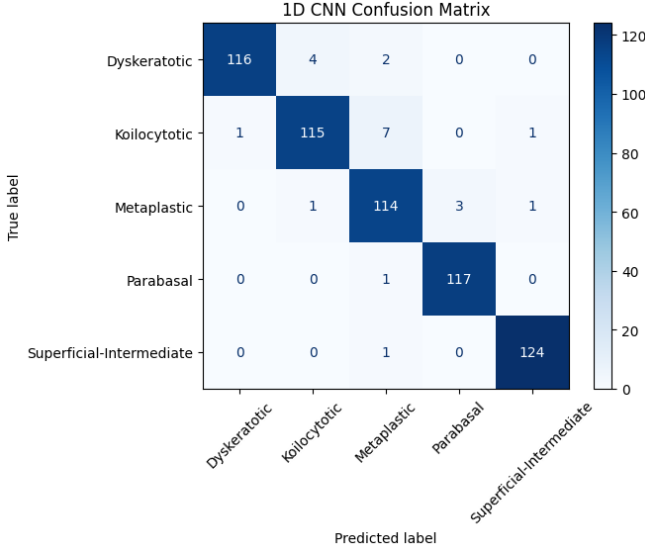


Fig. 22: Confusion matrix of the 1D CNN model using segmented nucleus-cytoplasm tabular features.

The Autoencoder + MLP model was used to test whether the 716-dimensional feature vector could be represented with fewer features. In this method, the autoencoder learned to reconstruct the original input features. Fig. 23 shows the reconstruction loss of the autoencoder. Both training and validation reconstruction losses decreased during training. This means that the autoencoder learned a compact representation of the segmented features.

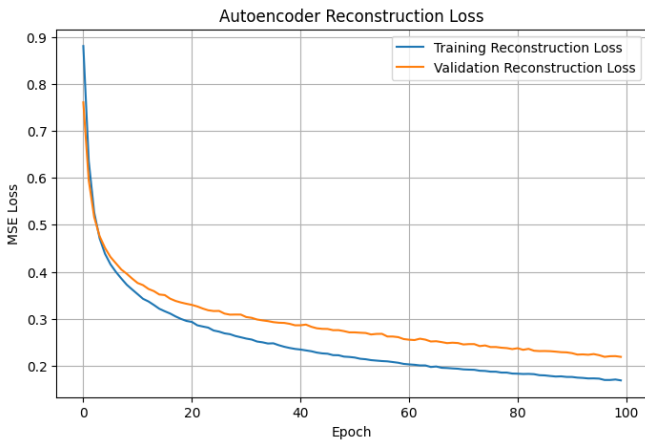


Fig. 23: Training and validation reconstruction loss curves of the autoencoder used for feature compression.

After the autoencoder, the encoded features were used to train an MLP classifier. Fig. 24 shows the loss curve of the

Autoencoder + MLP classifier. The training loss decreased steadily, and the validation loss also decreased at the beginning. After some epochs, the validation loss became higher than the training loss and stayed nearly stable. This shows that the encoded features were useful for classification, but the classifier learned the training set more strongly than the validation set. Therefore, a small overfitting tendency was observed. However, the final test accuracy was still above 96%, which shows that the model was still successful.

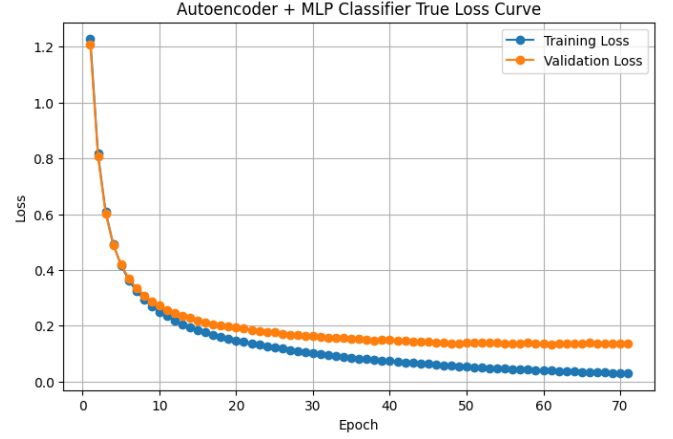


Fig. 24: True training and validation loss curves of the Autoencoder + MLP classifier.

Table XVIII shows the final comparison of the deep learning models. The Basic MLP model achieved the highest test accuracy with 96.71%. It also gave the highest macro F1-score with 0.9671. The 1D CNN model achieved 96.38% test accuracy. The Deep MLP model achieved 96.22% test accuracy. The Autoencoder + MLP model achieved 96.05% test accuracy.

TABLE XVIII: Performance comparison of deep learning models trained using segmented nucleus-cytoplasm features.

| Model             | Test Loss | Test Acc. | Precision | Recall | Macro F1 |
|-------------------|-----------|-----------|-----------|--------|----------|
| Basic MLP         | 0.2433    | 0.9671    | 0.9672    | 0.9673 | 0.9671   |
| 1D CNN            | 0.4203    | 0.9638    | 0.9642    | 0.9640 | 0.9638   |
| Deep MLP          | 0.1291    | 0.9622    | 0.9622    | 0.9623 | 0.9622   |
| Autoencoder + MLP | 0.1131    | 0.9605    | 0.9610    | 0.9605 | 0.9606   |

Overall, all tested models achieved test accuracies above 96%. This result shows that the segmented nucleus-cytoplasm features are useful for cervical cell classification. Among these models, the Basic MLP gave the best result. It also had a simpler structure than the deeper models. Therefore, the Basic MLP was selected as the best deep learning model for the segmented-feature part of this study. This result also shows that using nucleus and cytoplasm information can improve the classification process.

#### IV. CURRENT CONCLUSION

The current results show that segmentation improved the classification performance. In the non-segmented part, the best classical ML model was SVM with 94.90% accuracy, and the

best deep learning model was EfficientNetB3 with 94.57% accuracy.

After using segmented nucleus-cytoplasm features, the results increased. The best segmented classical ML model was Random Forest with 97.37% accuracy, and the best segmented deep learning model was Basic MLP with 96.71% accuracy.

These results show that nucleus and cytoplasm features are useful for separating the cell classes. Even though the segmentation was not perfect, it still helped the models learn more meaningful features.

## V. FUTURE WORK

The end goal of our project is to make the classification visible by using visualization tools such as GradCAM. To get GradCAM outputs, the model we use needs to get an image input and the models that do that are usually CNN models. We did use CNN models for not segmented images but for segmented images we still miss CNN type deep learning models. For our future work we aim to input segmented cell images for CNN types and at the end of the pipeline observe the model's focus while it classified the cell.

In short, we will follow the steps:

- Train CNN models which were used in non-segmented images for segmented images,
- Get GradCAM results,
- Overall compare all models tested.

We aim to see improved accuracy while completing the main objective of this project, visualizing the decision process.

## REFERENCES

- [1] A. Rosebrock, "Why is my validation loss lower than my training loss?," PyImageSearch, Oct. 14, 2019. [Online]. Available: <https://pyimagesearch.com/2019/10/14/why-is-my-validation-loss-lower-than-my-training-loss/>